

Documentação do Simulador UFLA-RISC

1. Resumo da Máquina Simulada

O simulador implementa o processador UFLA-RISC, uma arquitetura RISC de 32 bits com as seguintes características:

- **Pipeline:** 4 estágios funcionais (IF, ID, EX/MEM, WB).
- **Memória:** Endereçamento à palavra (word-addressed), com capacidade total simulada de 64k posições (256KB).
- **Registradores:** Banco com 32 registradores de uso geral (\$r0\$ a \$r31\$), onde \$r0\$ é fixo em zero e \$r31\$ é usado para link de retorno (\$JAL\$).
- **ALU:** Unidade Lógica e Aritmética que gera flags de condição (Zero, Negativo, Carry, Overflow).

2. Decisões de Implementação

O simulador foi desenvolvido em Python devido à facilidade de manipulação de listas e estruturas de dicionário para simular a memória e os registradores de pipeline.

- **Arquitetura Modular:** O código foi dividido em módulos (cpu, memory, registers, instruction, assembler) para facilitar a manutenção e testes unitários.
- **Simulação de Pipeline:** Utilizamos a classe PipelineRegister para simular os "latches" entre estágios. A função step() calcula o próximo estado de todos os estágios antes de atualizar os latches, garantindo o comportamento concorrente do hardware.
- **Tratamento de Hazards:**
 - **Dados:** Implementado via *Forwarding* (adiantamento) do estágio WB e EX/MEM para o início do EX.
 - **Load-Use:** Implementado via detecção de conflito no ID, inserindo uma "bolha" (NOP) e travando o PC por um ciclo.
 - **Controle:** Branches tomados provocam o *flush* (limpeza) das instruções nos estágios IF e ID anteriores.
- **Compatibilidade de Endereçamento:** Embora o simulador armazene dados palavra-por-palavra (índices 0, 1, 2...), a interface aceita endereços de byte (0, 4, 8...) incrementando o PC em 4, para manter a semelhança com arquiteturas reais como MIPS.

3. Instruções Projetadas pelo Grupo

Para cumprir o requisito de extensibilidade, foram adicionadas 8 novas instruções ao conjunto base, divididas em R-Type e I-Type.

R-Type

- **MUL rc, ra, rb:** Multiplicação inteira. *Justificativa:* Operação essencial não presente no set básico.
- **DIV rc, ra, rb:** Divisão inteira. *Justificativa:* Complemento aritmético básico.

- **NOR rc, ra, rb:** Operação lógica NOR. *Justificativa:* Permite a completude lógica funcional (universal gate).

I-Type

O formato adotado foi híbrido para permitir imediatos de 16 bits: Opcode(8) | Rt(4) | Rs(4) | Imm(16).

- **ADDI rt, rs, imm:** Adição com imediato.
- **SUBI rt, rs, imm:** Subtração com imediato.
- **MULI rt, rs, imm:** Multiplicação com imediato.
- **ANDI rt, rs, imm:** AND lógico com imediato.
- **SLTI rt, rs, imm:** Set Less Than Immediate.

Justificativa Geral: Instruções com imediatos reduzem a necessidade de carregar constantes da memória ou usar múltiplos registradores para contadores e flags simples.

4. Tutorial de Uso

O projeto inclui um montador automático integrado ao simulador.

1. **Pré-requisitos:** Python 3.x instalado.

Execução Interativa:

Na raiz do projeto, execute o comando abaixo para abrir o menu de seleção de testes:

2. `python3 -m src.simulador.main`

Execução Direta:

Para rodar um arquivo .asm específico:

`python3 -m src.simulador.main exemplos/teste_hazard.asm`

3. O sistema irá gerar automaticamente os binários na pasta binarios/ e iniciar a simulação.

5. Estruturas de Representação do Hardware

O hardware foi mapeado para estruturas de software da seguinte forma:

- **Banco de Registradores:** Lista de inteiros [0..31] dentro da classe RegisterFile.
- **Memória de Dados:** Lista dinâmica na classe DataMemory, simulando endereçamento alinhado.
- **Latches de Pipeline:** Dicionários Python (if_id_latch, id_ex_latch, etc.) contendo chaves para todos os sinais de controle (RegWrite, MemRead, etc.) e dados (PC, Instruction, ALU_Result).
- **ALU:** Método estático que recebe operandos e código de controle, retornando o resultado e um dicionário de Flags (Z, N, C, V).

6. Código Fonte

O código fonte completo encontra-se organizado na pasta `src/` do repositório, com a seguinte hierarquia:

- `src/simulador/`: Núcleo do processador (`cpu.py`), memória e decodificação.
- `src/interpretador/`: Montador (`assembler.py`) responsável por traduzir Assembly para código de máquina (Binário/Hex).

7. Testes Realizados

Foram desenvolvidos cenários de teste específicos para validar cada componente do pipeline :

1. **teste_add.asm / teste_mul.asm**: Validação da ALU, incluindo as novas instruções e cálculo de flags.
2. **teste_memoria.asm**: Verificação de operações LW e SW, garantindo a persistência de dados e o endereçamento correto.
3. **teste_hazard.asm**: Teste de estresse do pipeline. Força situações de dependência de dados (RAW) e *load-use*, validando se o forwarding e os stalls ocorrem corretamente.
4. **teste_jump_to_branch.asm**: Teste de fluxo de controle, verificando se saltos incondicionais (J) seguidos de condicionais (BEQ) limpam o pipeline corretamente sem executar instruções espúrias.